

Herramientas de IA

Preparación del repositorio y validación del starter

Fecha de corte: 2026-09-27. Documento de registro técnico en fase de desarrollo.

Equipo

Eduardo Valle | <https://linkedin.com/in/evallesv>

Jose Muñoz | <https://linkedin.com/in/jose-salcedo-442663293>

Santiago López | <https://linkedin.com/in/santiago-lopez-software-engineer>

Codex - En uso en la fase de desarrollo actual

Modelo: Astra 6

Propósito: Analiza las fuentes del reto y asiste en la estructuración de la arquitectura, contratos, código base y plan de implementación.

Aplicación: Interpreta las especificaciones del reto, genera código modular en Python y documentación técnica, y asiste en la verificación de pruebas locales.

Resultados: Provee la base modular en FastAPI, motor de cálculo determinista, fixtures sintéticos A-D, adaptador de Gemini, interfaz guiada, suite de pruebas automatizadas y runbook para Oracle ARM64.

Evidencia: README.md, app/, tests/, docs/implementation-plan.md, docs/verification.md

Límites: Opera como asistente de desarrollo en entorno local; el equipo humano revisa y valida cada cambio antes de publicarlo.

Fuentes analizadas

Especificaciones del reto 2; bases oficiales de hackIAthon Panamá; aviso de tratamiento de datos del organizador. Ubicaciones y hashes registrados en docs/sources/manifest.json.

Fecha de referencia de las bases: 23/09/2026. Este documento registra la base técnica desarrollada al 27/09/2026.

Integraciones y adaptadores

Las siguientes herramientas disponen de adaptadores y configuración integrados en el código, con activación progresiva según los requerimientos de cada tarea.

OpenCode - Seleccionado para desarrollo por tareas modulares

Modelo: Muse Spark 1.3

Propósito: Resuelve tareas modulares de implementación con agentes enfocados en optimización de contexto y bajo costo.

Aplicación: Cuenta con configuración e instrucciones preparadas para ejecutar las tareas modulares T01 a T08 de forma incremental.

Resultados: Estructura el plan de tareas con criterios de aceptación explícitos; no registra consumo de tokens en esta etapa inicial.

Evidencia: `opencode.json`, `AGENTS.md`, `docs/low-cost-agents.md`, `docs/tasks/`

Límites: Requiere registrar el consumo real de tokens y la versión exacta del modelo al momento de su ejecución.

Gemini API - Integrado mediante adaptador; simulación local activa

Modelo: Gemini 3.8 Flash

Propósito: Evalúa la consistencia semántica entre documentos del expediente y apoya la extracción asistida de datos.

Aplicación: Se comunica mediante el SDK oficial con function calling estricto y acotado, valida citas contra documentos activos y opera bajo modo de simulación segura sin exigir claves.

Resultados: Mantiene contratos verificados con pruebas automatizadas y respuestas simuladas; no realiza llamadas no autorizadas a la API externa.

Evidencia: `app/agent/provider.py`, `scripts/smoke_gemini.py`, `tests/test_provider.py`

Límites: No almacena credenciales en el repositorio; requiere configurar la clave de API en variables de entorno locales antes de su activación.

Control de calidad y seguridad

Cálculos con precisión Decimal; evidencia trazable y referenciada; revisión y decisión bajo criterio humano; datos de prueba sintéticos; credenciales fuera de Git. La arquitectura monolítica en Oracle ARM64 está preparada para despliegue reproducible.

Al completar la entrega: registrar commit de release, modelos definitivos, consumo medido y enlace público verificado; actualizar `tools-used.json` y regenerar este PDF.